

머신러닝 프로그래밍

Machine Learning Programming

Lecture 11 : Dimension Reduction



충북대학교
CHUNGBUK NATIONAL UNIVERSITY



충북대학교
산업인공지능연구센터

Dimension

차원 (Spatial Dimension)

일반적으로 생각하는 차원

공간의 정보를 결정하기 위한 축의 수

인공지능에서의 차원

대상을 특정하기 위한 **정보의 개수**

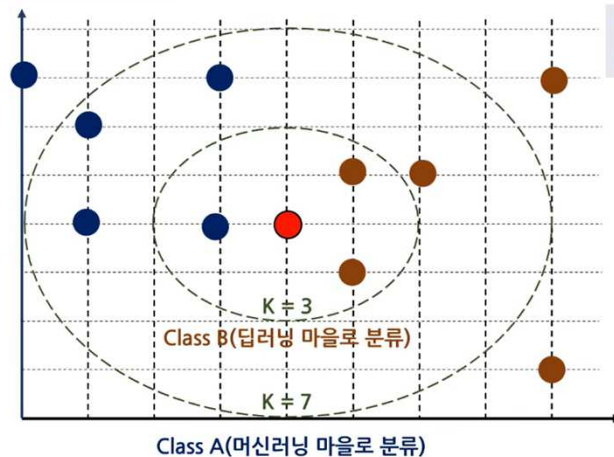
Dimension

공간으로 보는 차원 (Spatial Dimension)

- 0차원 (점): 위치만 존재, 크기나 방향이 없음
- 1차원 (선): '길이(x축)'라는 하나의 축으로 위치 설명
- 2차원 (면): '가로(x축)', '세로(y축)' (예: 평면 모니터 화면)
- 3차원 (공간): '가로', '세로', '높이(z축)' 세 개의 축.

K-Nearest Neighbor

2개의 정보로 대상을 표현 $\Rightarrow$ 2차원



Q) 새롭게 생긴
빨간 집은 어느 마을로 분류?

Dimension

벡터로 보는 차원 (Vector Dimension)

벡터 안에 포함된 원소(element)의 개수

1차원 벡터 (원소가 1개)

2차원 벡터 (원소가 2개)

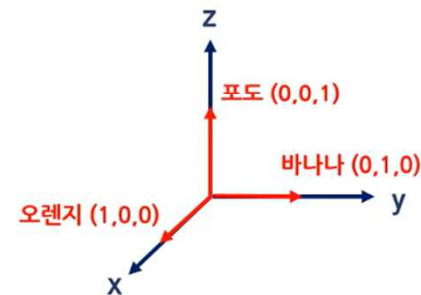
4차원 벡터 (원소가 4개)

3개의 정보로 대상을 표현 $\Rightarrow$ 3차원

One-Hot Encoding

	오렌지	바나나	포도
오렌지	1	0	0
바나나	0	1	0
포도	0	0	1

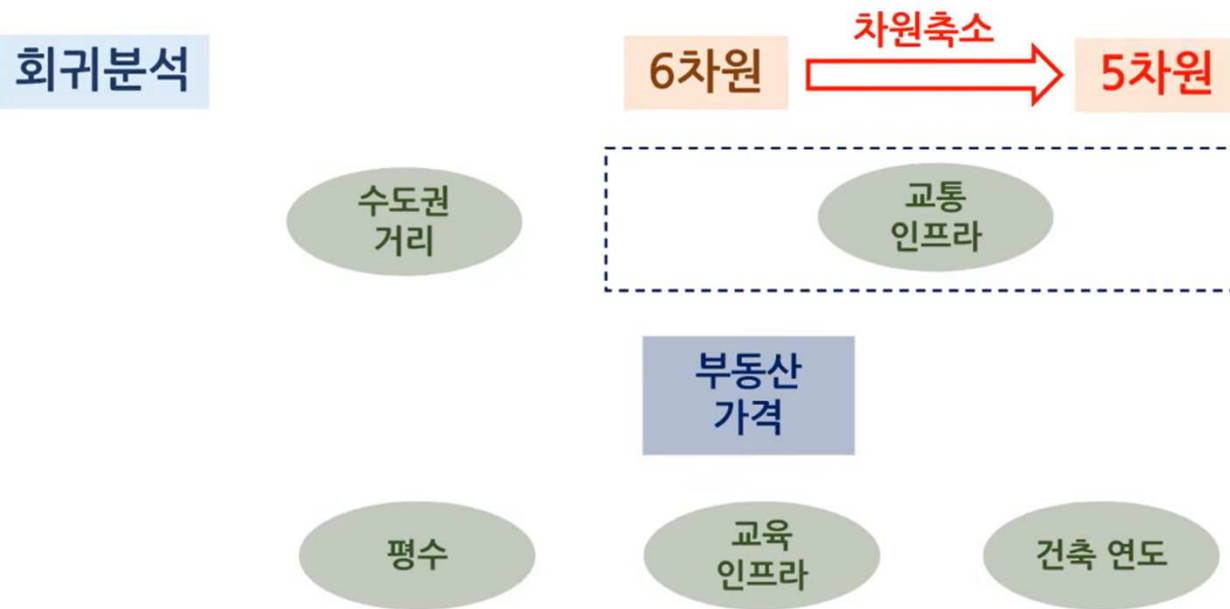
One-Hot
Encoding



Dimension

특성으로 보는 차원 (Feature Dimension)

- 데이터 분석과 머신러닝 실무에서 가장 흔하게 말하는 '데이터의 차원'
- 표 형태(Table)의 데이터에서 '특성(Feature, Column)의 개수'

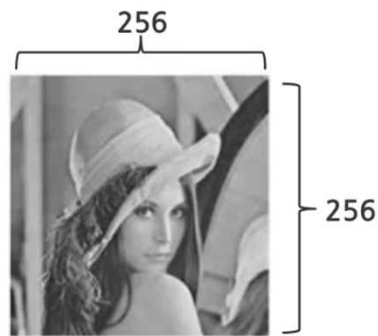


Dimension

이미지로 보는 차원 (Image Dimension)

2차원 텐서 (흑백 이미지): 높이(Height) x 너비(Width)의 2차원 행렬 구조.
(예: 픽셀의 손글씨 이미지)

3차원 텐서 (컬러 이미지): 높이(Height) x 너비(Width) x 채널(Channel)의 3차원 구조. 보통 Red, Green, Blue(RGB) 3개로 이루어짐.



(256X256) 차원



(256X256) 차원



(3X256X256) 차원

Red Channel



Green Channel



Blue Channel

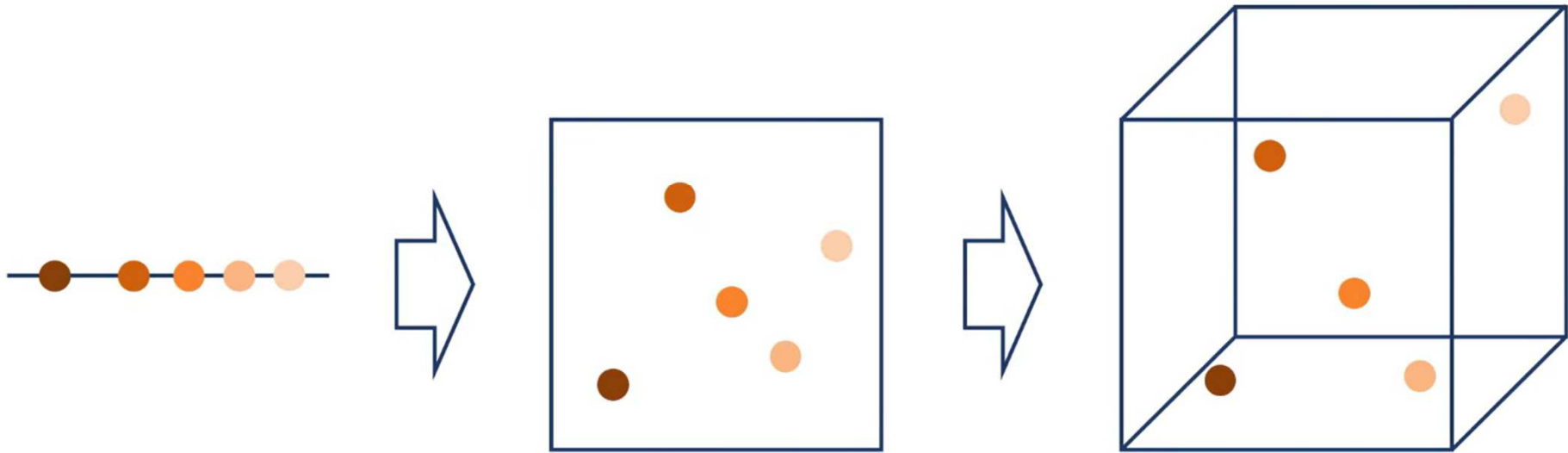


Curse of Dimensionality

차원의 저주 (Curse of Dimensionality)

데이터 학습 시 차원이 높아질수록 알고리즘의 성능이 저하되는 현상

수집된 데이터의 수 대비 분석해야 할 특성의 수가 많아질수록 의미 없는 정보 증가



해결방안

밀도가 높아지도록 충분한 데이터를 수집

차원이 높을수록 필요한 데이터 수가 기하급수적으로 증가

차원을 축소하여 특성의 수를 줄임

Dimension Reduction

차원 축소는 단순히 데이터를 가볍게(속도) 만드는 것을 넘어, 모델이 더 똑똑하게(성능) 학습하도록 돕는 필수 과정

- 속도뿐만 아니라 '성능' 향상을 위해 필수적
 - 속도 향상: 모델 학습에 불필요한 피쳐 제거 (연산량 감소)
 - 성능 향상: 모델 학습을 방해하는 피쳐 제거 (정확도 상승)

Dimension Reduction

차원 축소 방법으로 크게 형상 선택과 형상 추출 두가지 사용

1. 형상 선택 (Feature Selection)

중요한 피처만 골라냅니다 (Keep Important Features)

과일 가격	과일 색상	과일 크기	유통 거리	수집 날짜
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...

원본 고차원 데이터



선택된 핵심 피처

	과일 크기
과일 가격	...
...	...

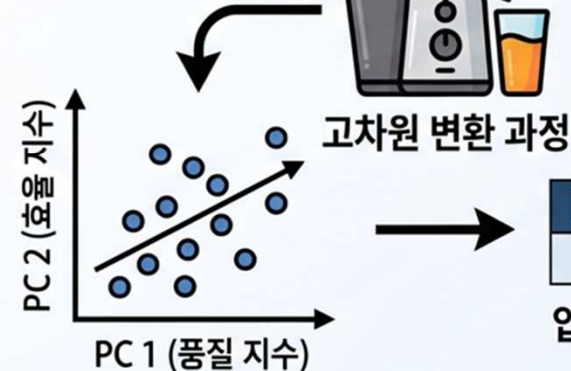
선택된 저차원 데이터

불필요한 피처 삭제 및 보존

2. 형상 추출 (Feature Extraction)

새로운 잠재 특성을 만듭니다 (Create New Latent Features)

과일 가격	과일 색상	과일 크기	유통 거리	수집 날짜
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...



PC 1	PC 2	PC 3
...	...	...

압축된 저차원 데이터

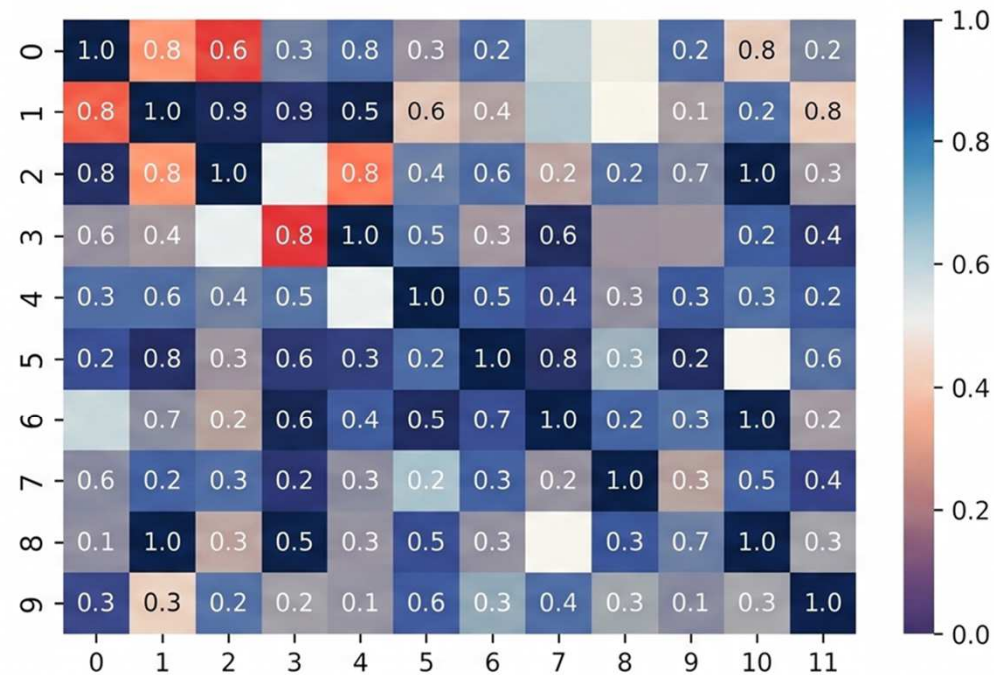
새로운 변환 피처 생성 및 압축

Feature Selection

- 형상 선택 (feature selection)
 - 종속 변수와 가장 관련성이 높은 피처만을 선택해, 나머지를 제외시킴
(각각의 피처를 모델에 포함시킴으로써 손실값이 낮아지는 정도 비교)
 - (Remind) Bagged tree에서의 피처 중요도 계산

Feature Selection

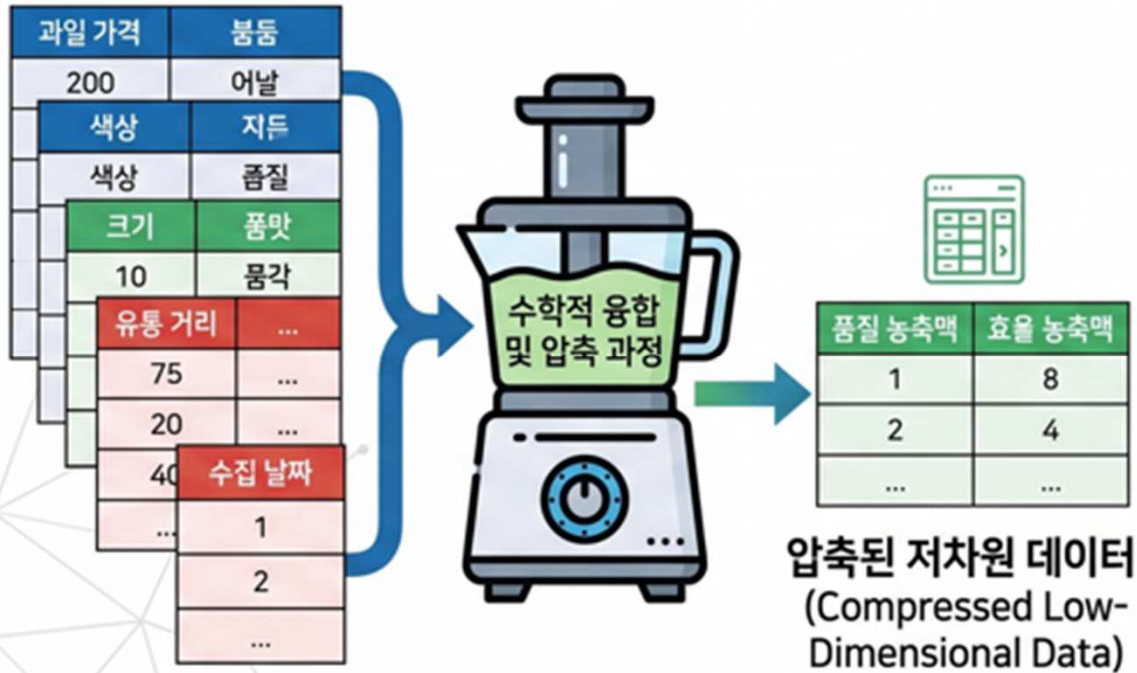
- 피처 사이의 **상관 관계가 매우 높아서**, 한 쪽이 의미 없는 경우 제외시킴
- 일반적으로 **히트맵(heatmap)**을 통해 여러 피처의 **공분산(covariance)** 분석



상관 관계 히트맵 예시 (Correlation Heatmap Example)

Feature Extraction

원본 고차원 데이터
(Original High-Dimensional Data)



형상 추출 (feature extraction)

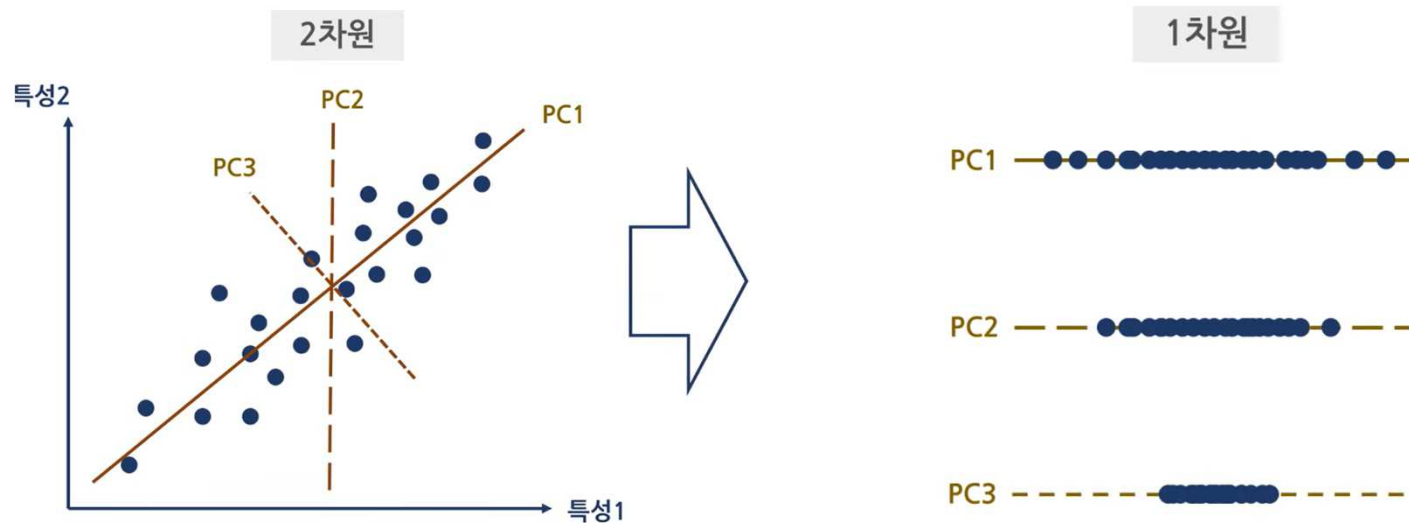
- 개별 피처를 제거하는 대신, **저차원 공간으로 투영시켜 데이터와 모델을 단순화**시킴
- 다음과 같은 알고리즘들이 있음
 1. PCA
 2. LDA
 3. MDS
 4. t-SNE

'채반(선택)' 대신 '믹서기(추출): 모든 피처를 갈아 새로운 특성 생성

Dimension Reduction

PCA (주성분 분석, Principal Component Analysis)

- 데이터 분포의 분산이 최대가 되는 정보로 차원을 축소시키는 방법
- 데이터의 원래 형태를 최대한 잃지 않기 위해, 데이터의 분산(Variance)을 최대한 보존하는 새로운 축(주성분)을 찾아서 그 축으로 데이터를 투영(Projection)
- 정답(Label)이 없는 상태에서 데이터 자체의 분포만 보고 축소(**비지도 학습**)



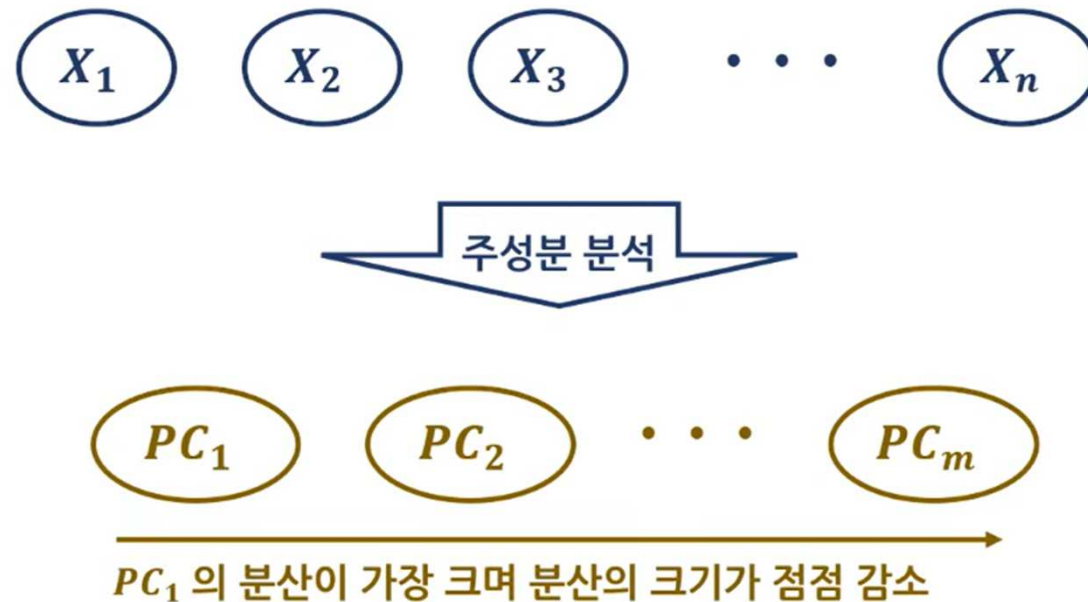
PC1이 가장 정보가 적게 손실되며 축소되는 주성분

Dimension Reduction

PCA (주성분 분석, Principal Component Analysis)

몇 개의 PC를 선택하는 것이 좋은가?

= 몇 차원으로 축소하는 것이 좋은가?



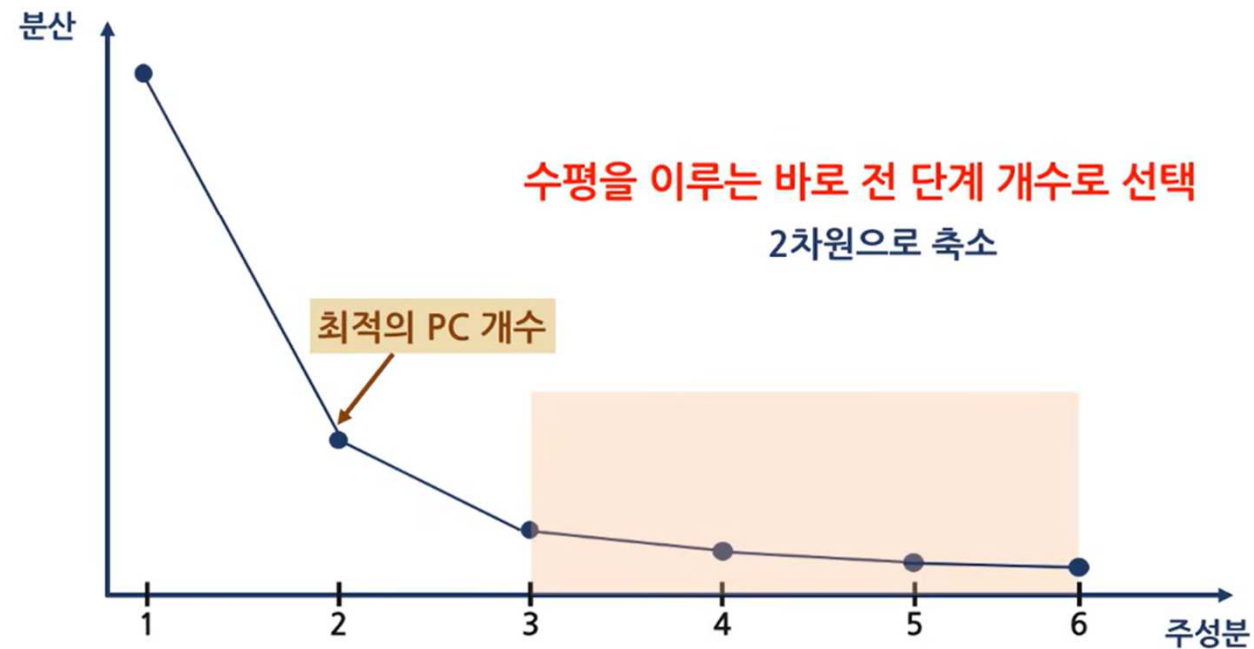
Dimension Reduction

PCA (주성분 분석, Principal Component Analysis)

몇 개의 PC를 선택하는 것이 좋은가?

스크리플롯(ScreePlot)

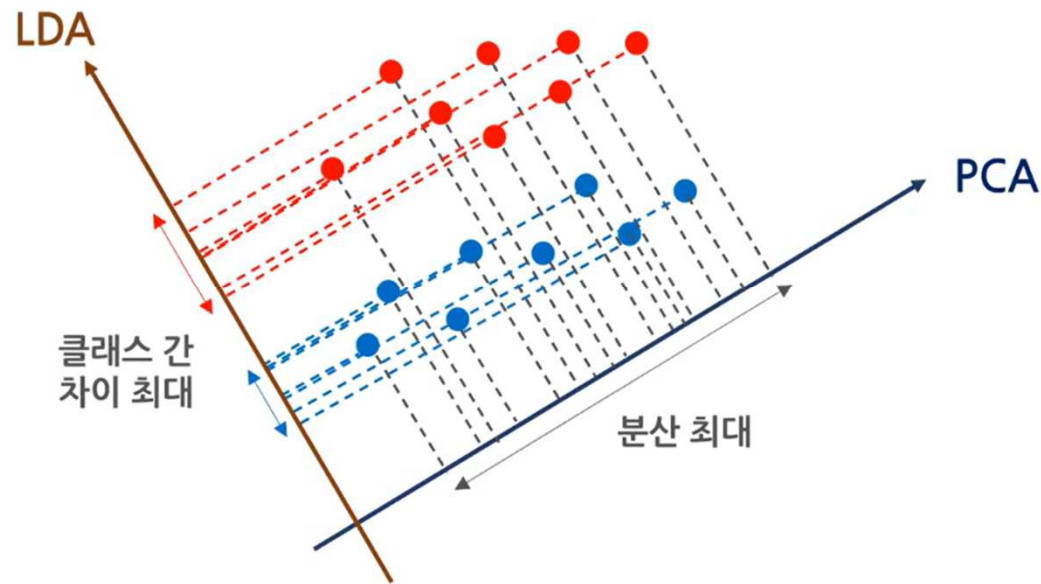
최적의 PC 개수를 찾기 위한 주성분 개수와 분산을 비교한 그래프



Dimension Reduction

LDA (선형 판별 분석, Linear Discriminant Analysis)

- 데이터의 분류정보를 활용해 클래스 간 차이를 최대화 하는 기법
- 데이터를 잘 표현하는 것이 아니라, 데이터를 **잘 분류하는 것**에 목적
- 같은 클래스끼리는 최대한 뭉치게 하고, 다른 클래스끼리는 최대한 멀리 떨어지게 하는 새로운 축을 찾음.
- 정답(Label) 데이터를 활용하여 차원을 축소(지도 학습)



Dimension Reduction

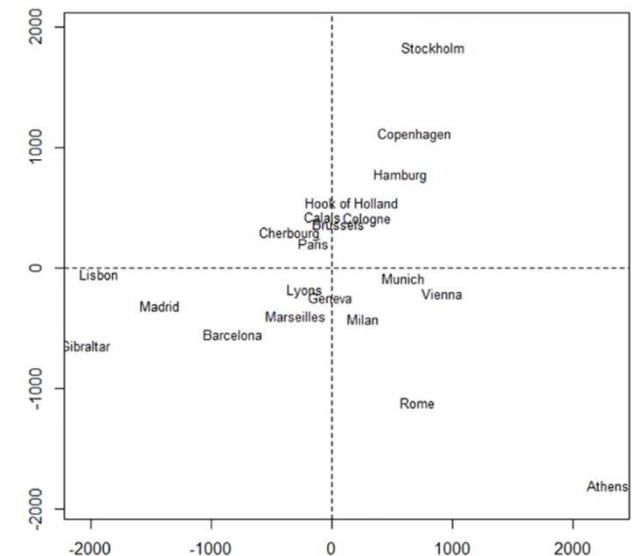
MDS (다차원 척도법, Multidimensional Scaling)

- 고차원 공간에서의 객체간 거리를 저차원에서도 최대한 유지하는 기법
- "A와 B는 가깝고, A와 C는 멀다"라는 원본 고차원 데이터의 **거리 및 유사도 구조를 2차원이나 3차원에 최대한 똑같이 재현하려고 노력**
- 변수 간의 관계나 데이터의 전반적인 구조를 시각적으로 탐색하고 싶을 때 주로 사용

유럽 20개 도시들간 거리 자료

	Athens	Barcelona	Brussels	Calais	Cherbourg	Cologne
Barcelona	3313					
Brussels	2963	1318				
Calais	3175	1326	204			
Cherbourg	3339	1294	583	460		
Cologne	2762	1498	206	409	785	
Copenhagen	3276	2218	966	1136	1545	760

MDS 활용 2차원 변환

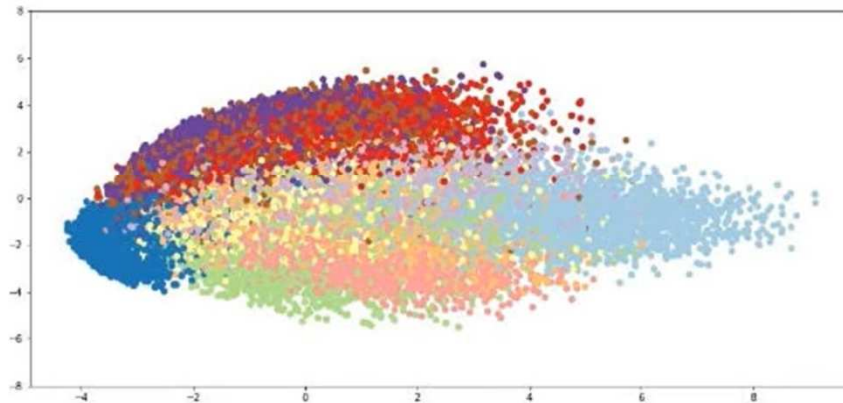


Dimension Reduction

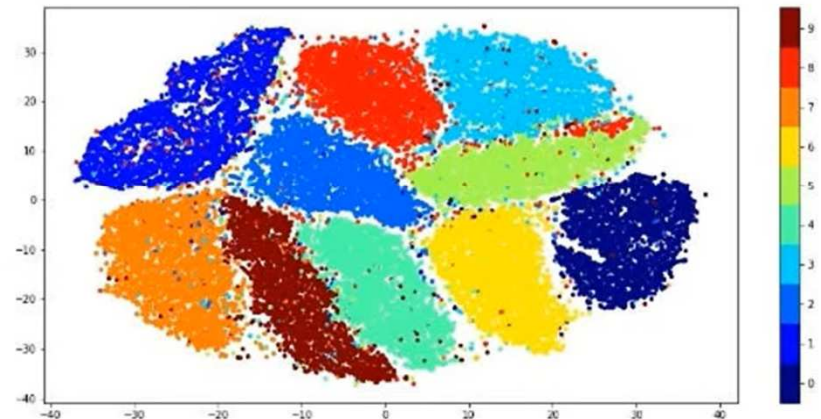
t-SNE (t-분포 확률적 임베딩, t-Distributed Stochastic Neighbor Embedding)

- 데이터간의 국소적 유사성을 보존하여 군집구조를 시각화하는 기법
- 고차원 공간에서 가까이 있는 이웃 데이터들이 저차원 공간에서도 가까이 있도록 확률적인 방식을 사용
- 데이터의 지역적 구조(Local Structure)를 보존하는 데 탁월

MNIST : PCA



MNIST : t-SNE



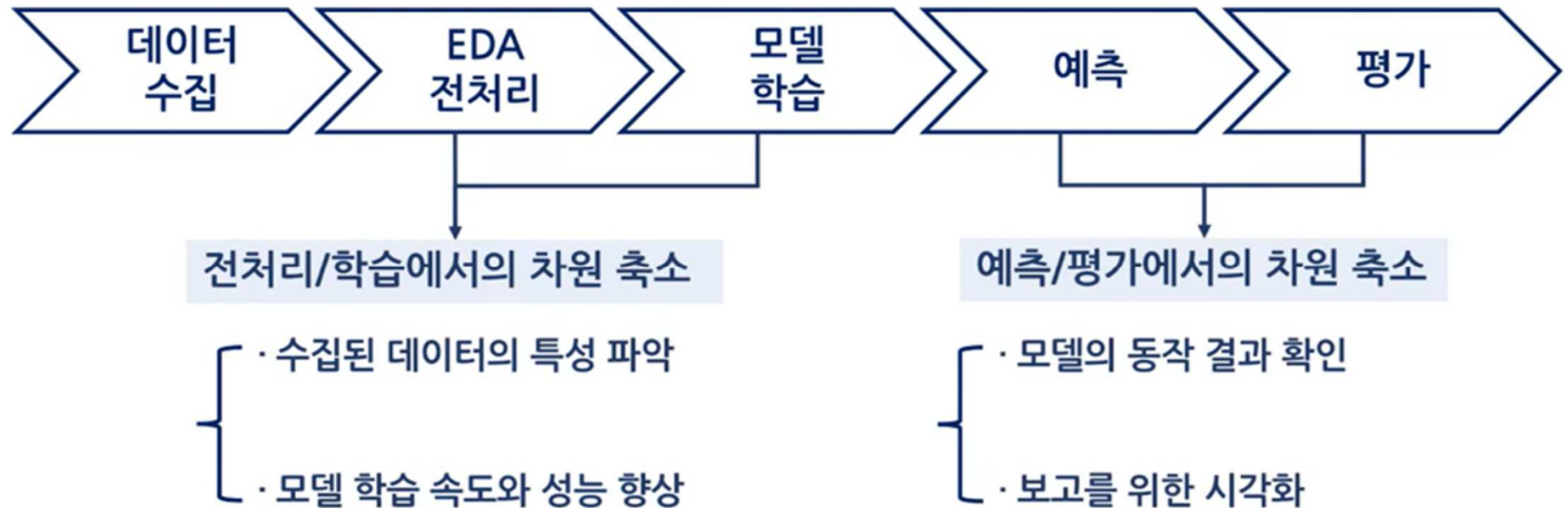
t-SNE의 속도 한계와 국소 구조의 편향을 해결하기 위해 **UMAP**이 등장

Dimension Reduction

- 데이터 자체의 형태를 최대한 보존하고 싶다면?
- 정답이 있고, 분류를 더 잘하게 만들고 싶다면?
- 데이터 간의 전체적인 거리/유사도 관계를 보고 싶다면?
- 복잡한 데이터를 2D/3D로 예쁘게 시각화해서 군집을 보고 싶다면?

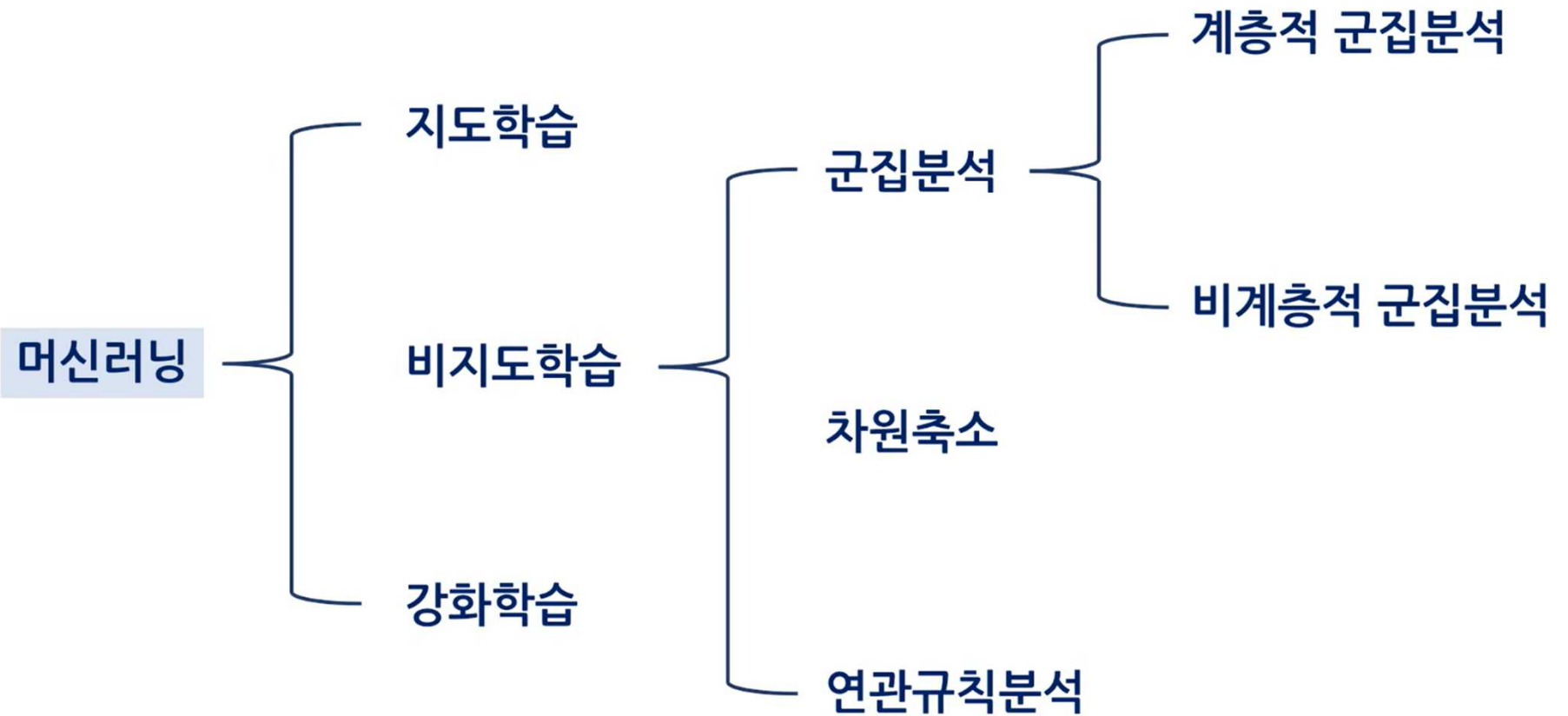
Dimension Reduction

차원 축소의 활용



Dimension Reduction

■ 비지도학습의 유형



실습

Step 1: 형상 선택 (Feature Selection) 11-1. 상관관계 히트맵

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
5 # 0. 데이터 불러오기 및 실전 전처리
6 file_path = r"D:\My_project\11 weeks\data.csv"
7
8 df = pd.read_csv(file_path)
9 print(f"원본 데이터 불러오기 성공! (크기: {df.shape})")
10
11 # 불필요한 ID 컬럼 및 빈 컬럼 제거
12 df = df.drop(labels=['id', 'Unnamed: 32'], axis=1, errors='ignore')
13
14 # 정답(diagnosis) 문자형 데이터를 숫자형(0과 1)으로 인코딩
15 df['target'] = df['diagnosis'].map({'B': 0, 'M': 1})
16 df = df.drop('diagnosis', axis=1)
17
18 # 피처(X)와 정답(y) 분리
19 X = df.drop('target', axis=1)
20 y = df['target']
21
22 print(f"✅ 전처리 완료! (피처 개수: {X.shape[1]}개)\n")
23
24 # Step 1: 형상 선택 (Feature Selection)
25 print("📊 상관관계 히트맵 시각화를 시작합니다...")
26
27 # 시각화 사이즈 설정
28 plt.figure(figsize=(12, 10))
29
30 # 30개 피처가 너무 많으니, 앞의 10개 피처만 잘라서 상관관계 계산
31 corr_matrix = X.iloc[:, :10].corr()
32
33 # Seaborn 라이브러리로 히트맵 그리기
34 sns.heatmap(corr_matrix,
35             annot=True,          # 칸 안에 숫자(상관계수) 표시
36             fmt=".2f",          # 소수점 둘째 자리까지 표시
37             cmap='coolwarm',    # 색상 테마 (빨간색일수록 양의 상관관계가 높음)
38             vmin=-1, vmax=1)    # 최소/최대값 설정
39
40 plt.title("Feature Correlation Heatmap (Top 10 Features)")
41 plt.show()
```

실습

데이터셋에 있는 변수 변경

```
# 30개 피처가 너무 많으니, 앞의 10개 피처만 잘라서 상관관계 계산  
corr_matrix = X.iloc[:, :10].corr()
```



1.

```
# 11번째부터 20번째 피처까지 10개를 자르고 싶을 때 (인덱스는 0부터 시작하므로 10:20)  
corr_matrix = X.iloc[:, 10:20].corr()
```
2.

```
# 내가 관심 있는 피처 이름만 리스트로 묶어서 쏙쏙 뽑아올 때  
my_features = ['radius_mean', 'texture_mean', 'smoothness_mean', 'symmetry_mean']  
corr_matrix = X.loc[:, my_features].corr()
```
3.

```
# 1. 모든 피처의 분산(Variance)을 계산하고, 내림차순(큰 것부터)으로 정렬하여 상위 10개 추출  
top10_variance_features = X.var().sort_values(ascending=False).head(10).index  
  
# 2. 골라낸 상위 10개 피처의 이름 확인해보기  
print("🔥 변동(분산)이 가장 큰 Top 10 피처:")  
print(top10_variance_features.tolist())  
  
# 3. 이 똑똑하게 골라낸 10개의 피처만 가지고 상관관계 히트맵 그리기  
corr_matrix = X[top10_variance_features].corr()
```


11-2. 피쳐 중요도

```

43 # -----
44 # 1-2. 트리 모델의 피쳐 중요도 (정답 힌트 점수 매기기)
45 from sklearn.ensemble import RandomForestClassifier
46
47 print("\n 🌳 1-2. 랜덤 포레스트를 활용한 피쳐 중요도 분석을 시작합니다.")
48
49 # 랜덤 포레스트 모델 생성 및 전체 피쳐(X)를 넣어 학습
50 rf_model = RandomForestClassifier(random_state=42)
51 rf_model.fit(X, y)
52
53 # 피쳐 중요도(Feature Importances) 추출 및 보기 좋게 시리즈로 변환
54 importances = pd.Series(rf_model.feature_importances_, index=X.columns)
55 top10_importances = importances.sort_values(ascending=False).head(10)
56
57 # 중요도가 높은 상위 10개 시각화 (막대그래프)
58 plt.figure(figsize=(10, 6))
59 top10_importances.plot.barh(color='steelblue')
60 plt.title("1-2. Feature Importance from Random Forest (The 'Strainer')")
61 plt.xlabel("Importance Score (Loss Reduction)")
62 plt.gca().invert_yaxis() # 1등이 맨 위에 오도록 y축 뒤집기
63
64 plt.tight_layout() # 글씨 잘림 방지
65 plt.show()
66
67 # 막대그래프 창을 닫으면 콘솔에 결과 표 출력
68 print("\n 📊 [결과 요약] 상위 10개 핵심 피쳐 및 중요도 점수")
69 print("-" * 50)
70
71 # 보기 좋은 표 형태로 변환하여 출력
72 top10_df = top10_importances.reset_index()
73 top10_df.columns = ['Feature Name (피쳐명)', 'Importance Score (중요도)']
74 print(top10_df.to_string(index=False))
75
76 print("-" * 50)
77 print("위 변수들이 정답을 맞히는 데 가장 큰 기여")

```


11-3. 형상추출 PCA_2D

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.decomposition import PCA
6
7 # 1. 데이터 준비 및 전처리
8 df = pd.read_csv(r"D:\My_project\11 weeks\data.csv")
9 df = df.drop(labels=['id', 'Unnamed: 32'], axis=1, errors='ignore')
10 df['target'] = df['diagnosis'].map({'B': 0, 'M': 1})
11 X = df.drop(labels=['diagnosis', 'target'], axis=1, errors='ignore')
12 y = df['target']
13
14 # 2. 형상 추출 (30차원 -> 2차원 압축)
15 X_scaled = StandardScaler().fit_transform(X) # 스케일링
16 pca = PCA(n_components=2)
17 X_pca = pca.fit_transform(X_scaled) # PCA 가동
18
19 # 3. 2차원 데이터 시각화 (그래프 내 한글 제거로 경고 완벽 해결)
20 pca_df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
21 pca_df['target'] = y.values
22
23 # 보존율 계산
24 pc1_ratio, pc2_ratio = pca.explained_variance_ratio_
25
26 plt.figure(figsize=(8, 5))
27 sns.scatterplot(data=pca_df, x='PC1', y='PC2', hue='target', palette='Set1', alpha=0.7)
28 plt.title("PCA Projection: 30D to 2D")
29 plt.xlabel(f"PC1 (Variance: {pc1_ratio:.1%})")
30 plt.ylabel(f"PC2 (Variance: {pc2_ratio:.1%})")
31 plt.show()
32
33 # 4. 결과 요약 출력 (콘솔은 한글 지원이 되므로 유지)
34 print(f"✓ PC1: 원본 정보의 {pc1_ratio:.1%} 보존")
35 print(f"✓ PC2: 원본 정보의 {pc2_ratio:.1%} 보존")
36 print(f"결론: 2개의 차원으로 원본 30차원 정보의 총 {pc1_ratio + pc2_ratio:.1%} 보존 완료!")
```

실습

11-4. 형상추출 PCA_3D

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.decomposition import PCA
5
6 # 1. 데이터 준비 및 전처리
7 df = pd.read_csv(r"D:\My_project\11 weeks\data.csv")
8 df = df.drop(labels=['id', 'Unnamed: 32'], axis=1, errors='ignore')
9 df['target'] = df['diagnosis'].map({'B': 0, 'M': 1})
10 X = df.drop(labels=['diagnosis', 'target'], axis=1, errors='ignore')
11 y = df['target']
12
13 # 2. 형상 추출 (30차원 -> 3차원 압축!)
14 X_scaled = StandardScaler().fit_transform(X)
15 pca = PCA(n_components=3) # ✨ 도착지를 3차원으로 변경!
16 X_pca = pca.fit_transform(X_scaled)
17
18 # 3. 3차원 데이터 시각화 준비
19 pca_df = pd.DataFrame(X_pca, columns=['PC1', 'PC2', 'PC3']) # ✨ PC3 기동 추가!
20 pca_df['target'] = y.values
21
22 # 보존율 계산
23 pc1_ratio, pc2_ratio, pc3_ratio = pca.explained_variance_ratio_
24
25 # ✨ 3D 도화지 펴기
26 fig = plt.figure(figsize=(10, 8))
27 ax = fig.add_subplot(111, projection='3d') # 3D 옵션 켜기
28
29 # 정답(target)별로 데이터를 분리해서 3D 공간에 점 찍기
30 df_0 = pca_df[pca_df['target'] == 0] # 양성
31 df_1 = pca_df[pca_df['target'] == 1] # 악성
32
33 ax.scatter(df_0['PC1'], df_0['PC2'], df_0['PC3'], color='steelblue', label='Benign (0)', alpha=0.7)
34 ax.scatter(df_1['PC1'], df_1['PC2'], df_1['PC3'], color='indianred', label='Malignant (1)', alpha=0.7)
35
36 # 축 이름과 제목 설정 (한글 폰트 에러 방지를 위해 영어 사용)
37 ax.set_title("PCA Projection: 30D to 3D")
38 ax.set_xlabel(f"PC1 ({pc1_ratio:.1%})")
39 ax.set_ylabel(f"PC2 ({pc2_ratio:.1%})")
40 ax.set_zlabel(f"PC3 ({pc3_ratio:.1%})") # ✨ Z축(깊이) 추가!
41 ax.legend()
42
43 plt.show()
44
45 # 4. 결과 요약 출력
46 print(f"✔ PC1: 원본 정보의 {pc1_ratio:.1%} 보존")
47 print(f"✔ PC2: 원본 정보의 {pc2_ratio:.1%} 보존")
48 print(f"✔ PC3: 원본 정보의 {pc3_ratio:.1%} 보존")
49 total_ratio = pc1_ratio + pc2_ratio + pc3_ratio
50 print(f"👉 결론: 3개의 차원으로 원본 30차원 정보의 총 {total_ratio:.1%} 보존 완료!")
```

실습

11-5. 형상추출 PCA_scee plot

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.decomposition import PCA
6
7 # 1. 데이터 준비 및 전처리
8 df = pd.read_csv(r"D:\My_project\11 weeks\data.csv")
9 df = df.drop(labels=['id', 'Unnamed: 32'], axis=1, errors='ignore')
10 df['target'] = df['diagnosis'].map({'B': 0, 'M': 1})
11 X = df.drop(labels=['diagnosis', 'target'], axis=1, errors='ignore')
12
13 # 2. 스케일링 및 전체 30차원 PCA 가동
14 X_scaled = StandardScaler().fit_transform(X)
15 pca = PCA()
16 pca.fit(X_scaled)
17
18 # 개별 보존율 및 누적 보존율 계산
19 variance_ratio = pca.explained_variance_ratio_
20 cumulative_variance = np.cumsum(variance_ratio)
21
22 # 3. 스크리 플롯(Scree Plot) 시각화 (스포일러 선 제거!)
23 plt.figure(figsize=(10, 6))
24 x_components = range(1, len(variance_ratio) + 1)
25
26 # 막대그래프 & 꺾은선 그래프
27 plt.bar(x_components, variance_ratio, alpha=0.5, color='steelblue', label='Individual Variance')
28 plt.plot(*args: x_components, cumulative_variance, marker='o', color='indianred', linewidth=2, label='Cumulative Variance')
29
30 plt.title("Scree Plot: Find the 'Elbow' Point!")
31 plt.xlabel("Number of Principal Components (PC)")
32 plt.ylabel("Explained Variance Ratio (0.0 to 1.0)")
33 plt.xticks(range(1, 32, 2))
34 plt.legend(loc='center right')
35 plt.grid(axis='y', linestyle='--', alpha=0.5)
36
37 plt.tight_layout()
38 plt.show()
39
40 # 4. 콘솔 결과 요약 출력 (질문형으로 변경!)
41 print("\n📌 [주요 PC 개수별 누적 정보 보존율]")
42 print("-" * 50)
43 print(f"✔️ PC 2개 사용 시: {cumulative_variance[1]:.1%} 보존")
44 print(f"✔️ PC 5개 사용 시: {cumulative_variance[4]:.1%} 보존")
45 print(f"✔️ PC 10개 사용 시: {cumulative_variance[9]:.1%} 보존")
46 print("-" * 50)
```

과제

11-6. 형상추출 _숙제

11-5. PCA_scree plot 를 진행하고 결과를 분석해서 차원축소를 진행하고
실습후 결과물 제출